

LAPORAN
WORKSHOP SISTEM INFORMASI WEB SERVER
(TIF330805)
MINGGU 5 PERTEMUAN 2

SEMESTER III



BKPM ACARA 10

Nama:
SEPTIAN DWI RAMADHAN PUTRA

NIM:
E41250282

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026

PENDAHULUAN

1. Latar Belakang

Pada Acara 9, pola Repository sudah diterapkan khusus untuk Mahasiswa: MahasiswaRepository menerima koneksi PDO lewat constructor, dan MahasiswaController menerima MahasiswaRepository dengan cara yang sama. Acara 10 menegaskan dan memperluas dua prinsip di balik pola tersebut secara eksplisit, yaitu Inheritance (pewarisan) pada Controller dan Model, serta Repository Pattern sebagai pemisah logika akses database dari logika aplikasi.

Tanpa Inheritance, setiap Controller (Home, Auth, Mahasiswa, Prodi, Matakuliah, Dosen) harus menulis ulang method `view()` untuk memuat tampilan dan `redirect()` untuk mengarahkan halaman. Tanpa Base Model, setiap Model yang mengakses tabel sendiri (Prodi, Matakuliah) harus membuat koneksi PDO masing-masing. Acara ini membuktikan bahwa aplikasi Sistem Informasi Akademik yang sudah dibangun sejak Acara 6 telah konsisten menerapkan kedua prinsip tersebut sejak awal, dan Acara 9 khusus menambahkan Repository Pattern untuk Mahasiswa.

2. Tujuan

1. Membuat BaseController yang berisi method `view()` dan `redirect()`.
2. Membuat MahasiswaController yang mewarisi (extends) BaseController dan menggunakan method yang diwariskan untuk menampilkan halaman data mahasiswa.
3. Membuat MahasiswaRepository yang menangani seluruh operasi database (tampil, tambah, ubah, hapus) data mahasiswa.
4. Memodifikasi MahasiswaController agar seluruh akses data dilakukan lewat MahasiswaRepository, tanpa query SQL di dalam Controller.
5. Menguji aplikasi: tampil data, tambah data, ubah data, dan hapus data, untuk memastikan seluruh fungsi tetap berjalan baik.

3. Dasar Teori Singkat

Inheritance (Pewarisan). Inheritance memungkinkan sebuah class mewarisi atribut dan method dari class lain, sehingga kode yang sama tidak perlu ditulis berulang. Pada aplikasi MVC, banyak Controller memiliki fungsi yang sama -- menampilkan halaman (`view()`) dan mengarahkan pengguna ke halaman lain (`redirect()`) -- sehingga dibuat satu BaseController yang diwarisi oleh seluruh Controller lain. Keuntungannya: mengurangi duplikasi kode (DRY), mempermudah pemeliharaan, dan perubahan cukup dilakukan pada BaseController tanpa menyentuh seluruh Controller satu per satu.

Repository Pattern. Repository Pattern adalah pola desain yang bertugas mengelola seluruh proses akses data ke database. Seluruh perintah SQL ditempatkan di dalam Repository, sehingga Controller hanya berinteraksi lewat method yang tersedia (`all()`, `find()`, `create()`, `dst.`) tanpa mengetahui bagaimana query dijalankan. Keuntungannya: memisahkan logika bisnis

dari logika akses database, Controller menjadi lebih sederhana, kode lebih mudah diuji, dan lebih mudah dikembangkan.

Base Model. Serupa dengan BaseController, Base Model menyediakan method umum (misalnya koneksi database) yang diwariskan oleh Model lain. Pada arsitektur ini, Base Model dipakai oleh Model yang masih bergaya Active Record sederhana (Prodi dan Matakuliah), sedangkan Mahasiswa -- karena sudah direfactor memakai Repository Pattern pada Acara 9 -- tidak lagi mewarisi Base Model, melainkan berperan sebagai Entity murni yang datanya dikelola MahasiswaRepository.

4. Alat dan Bahan

- Laptop/PC
- Web Browser
- Visual Studio Code
- XAMPP/Laragon
- PHP
- HTML, CSS, Bootstrap
- Modul Praktikum (BKPM Acara 10)

HASIL PRAKTIK

1. BaseController: app/Core/Controller.php

BaseController pada aplikasi ini sudah dibuat sejak Acara 6 dan dipertahankan tanpa perubahan perilaku. Seluruh Controller -- HomeController, AuthController, MahasiswaController, ProdiController, MatakuliahController, dan DosenController -- mewarisi class ini lewat extends Controller, sehingga method view() dan redirect() hanya ditulis satu kali.

```
1  <?php
2  |
3  abstract class Controller
4  {
5      protected function view(string $view, array $data = []): void
6      {
7          extract($data);
8          require __DIR__ . '/../Views/' . $view . '.php';
9      }
10
11     protected function redirect(string $path): void
12     {
13         redirect($path); // helper dari config/app.php
14     }
15 }
```

Bukti pewarisan pada MahasiswaController dan Controller lain (potongan deklarasi class):\

```
1  <?php
2  |
3  class MahasiswaController extends Controller
4  {
5      private MahasiswaRepository $repo;
```

```
1  <?php
2  |
3  class DosenController extends Controller
4  {
5      public function index(): void
6      {
7          $dosenList = Dosen::all();
8          $this->view('dosen/index', compact('dosenList'));
9      }
10 }
```

2. Base Model: app/Core/Model.php

Base Model menyediakan method `db()` yang mengambil satu koneksi PDO tunggal dari `Database::getInstance()` (singleton, dibuat pada Acara 8). Model Prodi dan Matakuliah mewarisi class ini, sehingga keduanya tidak membuat koneksi database sendiri-sendiri.

```
1 <?php
2 |
3 abstract class Model
4 {
5     protected static function db(): PDO
6     {
7         return Database::getInstance();
8     }
9 }
```

3. MahasiswaRepository: Memindahkan Seluruh SQL dari Controller

MahasiswaRepository (dibuat pada Acara 9, dipertahankan di sini) menampung seluruh query SQL terkait mahasiswa. MahasiswaController sama sekali tidak memuat SQL; ia hanya memanggil method milik Repository.

```

1 <?php
2
3 class MahasiswaRepository
4 {
5     private PDO $db;
6
7     public function __construct(PDO $db)
8     {
9         $this->db = $db;
10    }
11
12    // Daftar mahasiswa + nama prodi (JOIN), dengan pencarian opsional
13    // berdasarkan nama atau NIM (Tugas Mandiri Acara 8, dipertahankan di sini).
14    public function allWithProdi(?string $keyword = null): array
15    {
16        $sql = "SELECT m.*, p.nama AS prodi_nama
17                FROM mahasiswa m
18                JOIN prodi p ON m.prodi_id = p.id";
19
20        $params = [];
21        if ($keyword !== null && $keyword !== '') {
22            $sql .= " WHERE m.nama LIKE :kw_nama OR m.nim LIKE :kw_nim";
23            $params['kw_nama'] = '%' . $keyword . '%';
24            $params['kw_nim'] = '%' . $keyword . '%';
25        }
26        $sql .= " ORDER BY m.nim";
27
28        $stmt = $this->db->prepare($sql);
29        $stmt->execute($params);
30
31        return array_map([$this, 'hydrate'], $stmt->fetchAll());
32    }
33
34    // Seluruh mahasiswa beserta nama prodi (BKPM Acara 10: all()).
35    public function all(): array
36    {
37        return $this->allWithProdi(null);
38    }
39
40    public function find(int $id): ?Mahasiswa
41    {
42        $stmt = $this->db->prepare("SELECT * FROM mahasiswa WHERE id = :id");
43        $stmt->execute(['id' => $id]);
44        $row = $stmt->fetch();
45
46        return $row ? $this->hydrate($row) : null;
47    }
48
49    public function findByNim(?string $nim): ?Mahasiswa
50    {
51        if ($nim === null || $nim === '') {
52            return null;
53        }
54
55        $stmt = $this->db->prepare(
56            "SELECT m.*, p.nama AS prodi_nama
57            FROM mahasiswa m
58            JOIN prodi p ON m.prodi_id = p.id
59            WHERE m.nim = :nim"
60        );
61        $stmt->execute(['nim' => $nim]);
62        $row = $stmt->fetch();
63
64        return $row ? $this->hydrate($row) : null;
65    }
66
67    public function create(Mahasiswa $mhs): void
68    {
69        $stmt = $this->db->prepare(
70            "INSERT INTO mahasiswa (nim, nama, prodi_id, angkatan, status)
71            VALUES (:nim, :nama, :prodi_id, :angkatan, :status)"
72        );
73        $stmt->execute($this->toParams($mhs));
74    }
75
76    public function update(int $id, Mahasiswa $mhs): void
77    {
78        $params = $this->toParams($mhs);
79        $params['id'] = $id;
80
81        $stmt = $this->db->prepare(
82            "UPDATE mahasiswa
83            SET nim = :nim, nama = :nama,
84                prodi_id = :prodi_id, angkatan = :angkatan, status = :status
85            WHERE id = :id"
86        );
87        $stmt->execute($params);
88    }
89
90    public function delete(int $id): void
91    {
92        $stmt = $this->db->prepare("DELETE FROM mahasiswa WHERE id = :id");
93        $stmt->execute(['id' => $id]);
94    }
95
96    public function count(): int
97    {
98        return (int) $this->db->query("SELECT COUNT(*) FROM mahasiswa")->fetchColumn();
99    }
100
101    // Mengubah satu baris hasil query menjadi object Mahasiswa (Entity).
102    private function hydrate(array $row): Mahasiswa
103    {
104        $mhs = new Mahasiswa(
105            $row['nim'],
106            $row['nama'],
107            (int) $row['prodi_id'],
108            (int) $row['angkatan'],
109            $row['status']
110        );
111        $mhs->setId((int) $row['id']);
112
113        if (isset($row['prodi_nama'])) {
114            $mhs->setProdiNama($row['prodi_nama']);
115        }
116
117        return $mhs;
118    }
119
120    // Mengubah object Mahasiswa menjadi array parameter untuk PDO.
121    private function toParams(Mahasiswa $mhs): array
122    {
123        return [
124            'nim' => $mhs->getNim(),
125            'nama' => $mhs->getNama(),
126            'prodi_id' => $mhs->getProdiId(),
127            'angkatan' => $mhs->getAngkatan(),
128            'status' => $mhs->getStatus(),
129        ];
130    }
131 }
132

```

MahasiswaController memanggil method tersebut tanpa menuliskan satu baris SQL pun:

```
1  <?php
2
3  class MahasiswaController extends Controller
4  {
5      private MahasiswaRepository $repo;
6
7      public function __construct()
8      {
9          $this->repo = new MahasiswaRepository(Database::getInstance());
10     }
11
12     public function index(): void
13     {
14         $keyword      = trim($_GET['q'] ?? '');
15         $mahasiswaList = $this->repo->allWithProdi($keyword !== '' ? $keyword : null);
16         $prodiList     = Prodi::all();
17         $flash         = flash_get();
18
19         $this->view('mahasiswa/index', compact('mahasiswaList', 'prodiList', 'keyword', 'flash'));
20     }
```

```
46     public function store(): void
47     {
48         try {
49
50             $mhs = $this->buildEntityFromRequest();
51             $this->repo->create($mhs);
52             flash_set('success', 'Data mahasiswa berhasil ditambahkan.');
```

```
53             $this->redirect('mahasiswa');
54         } catch (InvalidArgumentException $e) {
55             $_SESSION['form_error'] = $e->getMessage();
56             $_SESSION['old_mahasiswa'] = $_POST;
57             $this->redirect('mahasiswa/create');
58         }
59     }
```

4. Hasil Pengujian: Tampil, Tambah, Ubah, Hapus

a) Tampil data -- GET /mahasiswa, lewat MahasiswaController::index() yang mewarisi view() dari BaseController.

SI Akademik Polije

Dashboard

Data Mahasiswa

Program Studi

Mata Kuliah

Data Dosen

Logout

Data Mahasiswa

Daftar mahasiswa yang terdaftar

Selamat datang, admin

Daftar Mahasiswa

7 data

Cari nama atau NIM...

+ Tambah Mahasiswa

NIM	NAMA	PRODI	ANGKATAN	STATUS	AKSI		
23004	Dewi	Teknik Informatika	2026	Aktif	Detail	Edit	Delete
2302001	Bagas Pradana	Teknik Informatika	2023	Lulus	Detail	Edit	Delete
2401001	Budi Santoso	Teknik Informatika	2024	Cuti	Detail	Edit	Delete
2401002	Ani Wijaya	Teknik Komputer	2025	Aktif	Detail	Edit	Delete
2401003	Asti	Teknik Informatika	2026	Aktif	Detail	Edit	Delete
2401004	Feri	Sistem Informasi	2026	Aktif	Detail	Edit	Delete

Gambar 1. Daftar mahasiswa sebelum pengujian (3 data awal dari seeding)

b) Tambah data -- POST /mahasiswa/store, data baru (Galih Pratama) berhasil masuk lewat MahasiswaRepository::create().

SI Akademik Polije

Dashboard

Data Mahasiswa

Program Studi

Mata Kuliah

Data Dosen

Logout

Data Mahasiswa

Daftar mahasiswa yang terdaftar

Selamat datang, admin

Daftar Mahasiswa

8 data

Cari nama atau NIM...

+ Tambah Mahasiswa

NIM	NAMA	PRODI	ANGKATAN	STATUS	AKSI
23004	Dewi	Teknik Informatika	2026	Aktif	DetailEditDelete
2301006	Galih Pratama	Teknik Informatika	2023	Aktif	DetailEditDelete
2302001	Bagas Pradana	Teknik Informatika	2023	Lulus	DetailEditDelete
2401001	Budi Santoso	Teknik Informatika	2024	Cuti	DetailEditDelete
2401002	Ani Wijaya	Teknik Komputer	2025	Aktif	DetailEditDelete

Gambar 2. Data baru berhasil ditambahkan, flash message “berhasil ditambahkan” tampil

c) Ubah data -- form edit terisi otomatis dari MahasiswaRepository::find(), lalu disimpan lewat update().

The screenshot shows a web application interface for managing student data. On the left is a dark sidebar with navigation links: Dashboard, Data Mahasiswa (highlighted), Program Studi, Mata Kuliah, Data Dosen, and Logout. The main content area is titled 'Data Mahasiswa' with a subtitle 'Daftar mahasiswa yang terdaftar'. A green success message at the top states 'Data mahasiswa berhasil diperbarui.' Below this is a table titled 'Daftar Mahasiswa' with 8 data items. The table has columns for NIM, NAMA, PRODI, ANGKATAN, STATUS, and AKSI. The data rows are as follows:

NIM	NAMA	PRODI	ANGKATAN	STATUS	AKSI
23004	Dewi	Teknik Informatika	2026	Aktif	Detail Edit Hapus
2301010	Galih Pratama	Teknik Informatika	2023	Aktif	Detail Edit Hapus
2302001	Bagas Pradana	Teknik Informatika	2023	Lulus	Detail Edit Hapus
2401001	Budi Santoso	Teknik Informatika	2024	Cuti	Detail Edit Hapus
2401002	Ani Wijaya	Teknik Komputer	2025	Aktif	Detail Edit Hapus

Gambar 3. Form ubah data Galih Pratama

d) Setelah diubah, detail menampilkan data terbaru (NIM berubah).

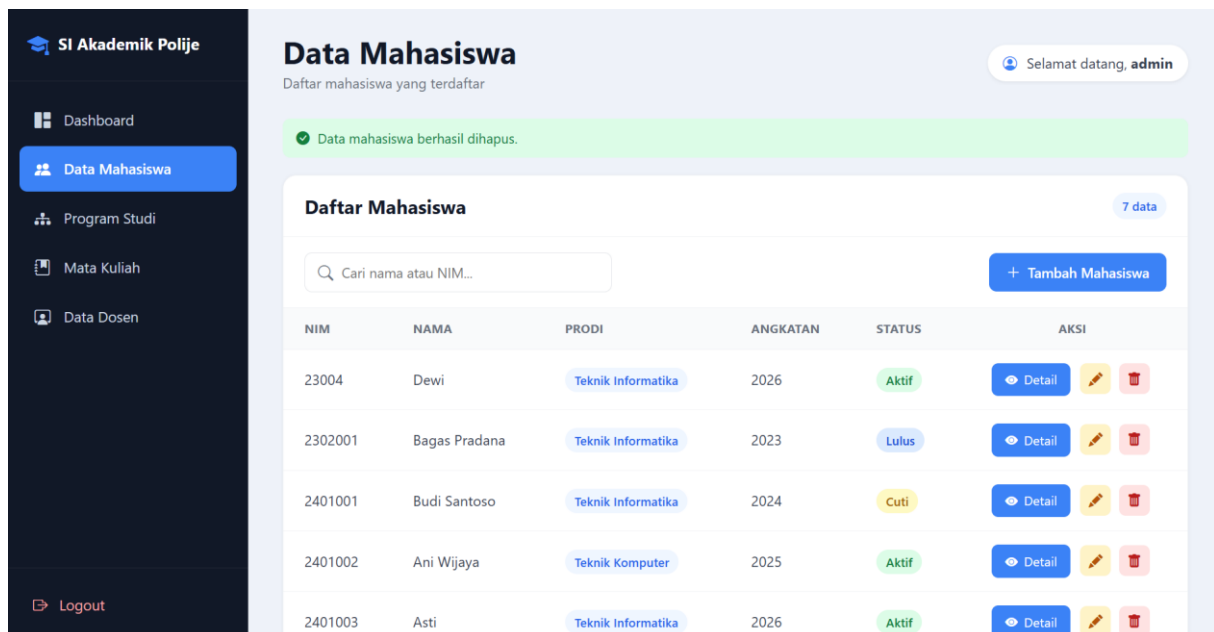
The screenshot shows the 'Detail Mahasiswa' page. It has a title 'Detail Mahasiswa' and a subtitle 'Informasi lengkap data mahasiswa'. The main content is a 'Profil Mahasiswa' card. Inside the card, the following information is displayed:

- NIM: 2301010
- Nama: Galih Pratama
- Program Studi: Teknik Informatika
- Angkatan: 2023
- Status: Aktif

At the bottom of the card are three buttons: '← Kembali', 'Ubah' (with an edit icon), and 'Hapus' (with a delete icon).

Gambar 4. Detail mahasiswa menampilkan hasil perubahan

e) Hapus data -- POST /mahasiswa/destroy, data Galih Pratama dihapus lewat MahasiswaRepository::delete(), daftar kembali menjadi 3 data.



Gambar 5. Data berhasil dihapus, flash message “berhasil dihapus” tampil, jumlah data kembali ke 3

5. Analisis dan Pembahasan

Pengujian pada langkah kerja butir 5 membuktikan bahwa seluruh fungsi CRUD mahasiswa (tampil, tambah, ubah, hapus) berjalan normal sepenuhnya lewat kombinasi Inheritance dan Repository Pattern: MahasiswaController tidak mewarisi logika akses data secara langsung, melainkan mewarisi method umum (view, redirect) dari BaseController, sedangkan logika akses data didelegasikan ke MahasiswaRepository yang diterima lewat constructor.

Prodi dan Matakuliah menjadi pembanding yang berguna: keduanya masih memakai pola Base Model (extends Model), yaitu method statis yang langsung menjalankan query di dalam Model itu sendiri. Pola ini lebih sederhana untuk data yang strukturnya jarang berubah, tetapi kurang fleksibel dibanding Repository Pattern karena logika akses data tetap menyatu dengan class Model, bukan dipisah ke class tersendiri yang dapat diganti implementasinya tanpa mengubah Model.

Kedua pendekatan Base Model untuk Prodi/Matakuliah, dan Repository Pattern untuk Mahasiswa hidup berdampingan dalam satu aplikasi yang sama, menunjukkan bahwa pemilihan pola desain dapat disesuaikan dengan kebutuhan masing-masing bagian aplikasi, bukan aturan yang harus seragam di seluruh kode.

KESIMPULAN

1. BaseController (app/Core/Controller.php) menyediakan method view() dan redirect() yang diwarisi oleh seluruh Controller, sehingga kedua method tersebut hanya ditulis satu kali (DRY).
2. MahasiswaController mewarisi BaseController lewat extends Controller, dan memakai method yang diwariskan tersebut pada setiap halaman data mahasiswa.
3. MahasiswaRepository menangani seluruh operasi database (tampil, tambah, ubah, hapus) data mahasiswa, menerima koneksi PDO lewat constructor.
4. MahasiswaController telah dimodifikasi sehingga seluruh akses data mahasiswa dilakukan lewat MahasiswaRepository, tanpa satu baris query SQL pun di dalam Controller.
5. Pengujian tampil, tambah, ubah, dan hapus data seluruhnya berhasil, membuktikan bahwa kombinasi Inheritance dan Repository Pattern membuat kode lebih rapi tanpa mengorbankan fungsionalitas aplikasi.

DAFTAR PUSTAKA

Buku Kerja Praktik Mahasiswa (BKPM) Workshop Sistem Informasi Web Server (TIF330805). Program Studi D4 Teknik Informatika PSDKU Nganjuk, Politeknik Negeri Jember.